Bag of Words y matrices de coocurrencia

Cómo convertimos texto en números y qué información conserva cada representación

Objetivo. Vamos a construir Bag of Words y matrices de coocurrencia desde sus ideas más sencillas hasta versiones que podríamos utilizar en un proyecto real. Nuestro énfasis será comprender qué representa cada fila, cada columna y cada número; después veremos cómo regex puede ayudar a limpiar y estructurar el texto antes de convertirlo en matrices.

En PLN solemos partir de algo muy humano -frases, comentarios, correos, mensajes- y necesitamos convertirlo en números para poder utilizar algoritmos de clasificación, agrupamiento, similitud o aprendizaje automático. One-Hot Encoding, Bag of Words y coocurrencia resuelven partes distintas de ese problema.

Texto → **Regex** → **Tokens** → **Vocabulario** → **Matriz** → **Modelo o análisis**

Primero conviene recordar la idea de One-Hot Encoding

Si el vocabulario contiene gato, perro y tarea, se puede asignar una posición a cada palabra. Cada palabra se representa con un 1 en su propia posición y 0 en las demás.

One-Hot: una palabra, una posición

Fila \ Columna	gato	perro	tarea
gato	1	0	0
perro	0	1	0
tarea	0	0	1

Qué sí indica. One-Hot permite distinguir la identidad de los términos. Sin embargo, por sí solo no indica cuántas veces aparece una palabra dentro de un documento ni con qué otras palabras suele aparecer.

Bag of Words: de palabras sueltas a documentos completos

Bag of Words, o BoW, representa cada documento mediante un vector de conteos. Para construirlo necesitamos acordar un vocabulario común. Después contamos cuántas veces aparece cada palabra del vocabulario en cada documento.

Una forma de imaginarlo. Se colocan todas las palabras de un documento en una bolsa. Al final no se conserva el orden exacto, pero sí se puede revisar la bolsa y contar cuántos gato, cuántos modelo o cuántos entrega había.

Primer ejemplo con tres documentos

Documento	Texto
D1	el gato come pescado
D2	el perro come carne
D3	el gato y el perro juegan

El procedimiento será siempre el mismo: normalizamos, tokenizamos, construimos un vocabulario sin palabras repetidas y finalmente llenamos la matriz documento por término.

Normalizar	→	Tokenizar	→	Crear vocabulario	→	Contar	→	Formar matriz

Tokenización

Documento	Tokens
D1	`[el, gato, come, pescado]`
D2	`[el, perro, come, carne]`
D3	`[el, gato, y, el, perro, juegan]`

Vocabulario

carne	come	el	gato	juegan	pescado	perro	y

Cada palabra aparece una sola vez en el vocabulario. El orden puede ser cualquiera, pero una vez fijado debemos conservarlo para todos los documentos.

Matriz BoW

Cada fila es un documento; cada columna es una palabra

Fila \ Columna	carne	come	el	gato	juegan	pescado	perro	y
D1	0	1	1	1	0	1	0	0
D2	1	1	1	0	0	0	1	0
D3	0	0	2	1	1	0	1	1

Por ejemplo, D3-el vale 2 porque la palabra 'el' aparece dos veces en D3. D1-perro vale 0 porque 'perro' no aparece en D1.

Segundo ejemplo: ahora con el salón de PLN

Documento	Frase
D1	el profe dijo regex y todos fingen entender
D2	mi modelo funciona hasta que lo presento
D3	si el código corre a la primera, sospecha: probablemente olvidaste probarlo

Qué aprender de este ejemplo. BoW no necesita frases solemnes. Si modelo aparece en un documento y no en los demás, su columna ayuda a distinguirlo. El chiste se pierde; los conteos se quedan.

Una evolución frecuente de BoW: TF-IDF

El BoW clásico trata cada aparición como un conteo. Eso puede dar demasiado peso a palabras muy frecuentes que aparecen en casi todos los documentos. Una evolución muy utilizada es TF-IDF: mantiene la estructura documento por término, pero pondera más los términos que son frecuentes en un documento y relativamente raros en el resto del corpus.

Intuición. Si una palabra aparece en todos los documentos quizá no ayude mucho a distinguirlos. Si aparece muchas veces en un documento concreto y rara vez en los demás, probablemente sea más informativa para ese documento. Más adelante podemos estudiar TF-IDF con detalle en un apunte propio.

Aspecto	BoW por conteo	TF-IDF
Valor de una celda	Número de apariciones.	Peso de importancia relativa.
Palabras muy comunes	Pueden dominar el vector.	Tienden a recibir menor peso.
Interpretación	Muy directa.	Menos inmediata que un conteo.
Orden de palabras	No lo conserva.	Tampoco lo conserva por sí solo.

Construcción de Bag of Words en Python

A continuación se programa sin una librería especializada de vectorización. Para retomar regex, se normalizará a minúsculas y se utilizará re.findall() para tokenizar. Se evitarán funciones de listas que oculten parte de la lógica; así el código se parece más a una implementación que puede trasladarse a otros lenguajes.

```
import re

documentos = [
    "El gato come pescado.",
    "El perro come carne.",
    "El gato y el perro juegan."
]

tokens_documentos = [None] * len(documentos)
indice_vocabulario = {}

i = 0

while i < len(documentos):

    texto = documentos[i].lower()

    tokens = re.findall(
        r"[a-záéíóúñ]+",
        texto
    )

    tokens_documentos[i] = tokens

    j = 0

    while j < len(tokens):

        palabra = tokens[j]

        if palabra not in indice_vocabulario:
            indice_vocabulario[palabra] = len(indice_vocabulario)

        j += 1

    i += 1

vocabulario = [None] * len(indice_vocabulario)

for palabra in indice_vocabulario:
    posicion = indice_vocabulario[palabra]
    vocabulario[posicion] = palabra

matriz = [
    [0] * len(vocabulario)
    for _ in range(len(documentos))
]

i = 0

while i < len(tokens_documentos):

    tokens = tokens_documentos[i]
    j = 0

    while j < len(tokens):
```

```
        palabra = tokens[j]
        columna = indice_vocabulario[palabra]
        matriz[i][columna] += 1

        j += 1

    i += 1

print("Vocabulario:")
print(vocabulario)

print("\nMatriz:")

i = 0
while i < len(matriz):
    print("D", i + 1, matriz[i])
    i += 1
```

Salida:

```
Vocabulario:
['el', 'gato', 'come', 'pescado', 'perro', 'carne', 'y', 'juegan']

Matriz:
D 1 [1, 1, 1, 1, 0, 0, 0, 0]
D 2 [1, 0, 1, 0, 1, 1, 0, 0]
D 3 [2, 1, 0, 0, 1, 0, 1, 1]
```

Aquí re.findall() cumple el papel de tokenizador. El diccionario indice_vocabulario evita duplicados y recuerda en qué columna debe caer cada palabra. Después se recorren los tokens y sumamos 1 directamente en la celda correspondiente.

Mensaje para PLN. Si al principio el vocabulario y las columnas parecen demasiados detalles, conviene seguir una regla sencilla: una columna = un término; una fila = un documento. Una vez dominada esa lectura, los modelos posteriores dejan de parecer magia y empiezan a verse como operaciones sobre datos.

Una versión más compacta y eficiente

La misma lógica puede conservarse la misma lógica pero usar Counter, una estructura de Python especializada en conteos. Se mantiene el uso de regex para tokenizar, pero ya no contamos manualmente cada repetición.

Versión con Counter

```
import re
from collections import Counter

documentos = [
    "El profe dijo regex y todos fingen entender.",
    "Mi modelo funciona hasta que lo presento.",
    "Si el código corre a la primera, sospecha: probablemente olvidaste probarlo."
]

tokens_documentos = [
    re.findall(
        r"[a-záéíóúñ]+",
        texto.lower()
    )
    for texto in documentos
]

conteos = [
    Counter(tokens)
    for tokens in tokens_documentos
]

indice = {}

for contador in conteos:
    for palabra in contador:
        if palabra not in indice:
            indice[palabra] = len(indice)
```

```
vocabulario = [None] * len(indice)

for palabra in indice:
    vocabulario[indice[palabra]] = palabra

matriz = [
    [0] * len(vocabulario)
    for _ in range(len(documentos))
]

i = 0

while i < len(conteos):

    contador = conteos[i]

    for palabra in contador:
        columna = indice[palabra]
        matriz[i][columna] = contador[palabra]

    i += 1

print(vocabulario)

for i in range(len(matriz)):
    print("D", i + 1, matriz[i])
```

Salida:

```
['el', 'profe', 'dijo', 'regex', 'y', 'todos', 'fingimos', 'entender', 'mi', 'modelo', 'funciona',
'hasta', 'que', 'lo', 'presento', 'si', 'código', 'corre', 'a', 'la', 'primera', 'sospecha',
'probablemente', 'olvidaste', 'probarlo']
D 1 [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
D 2 [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
D 3 [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Counter calcula los conteos de cada documento con una estructura optimizada. El programa sigue siendo transparente: tokenizamos, contamos, creamos el índice del vocabulario y llenamos la matriz.

Librerías que pueden sustituir esta implementación. Para proyectos reales podemos usar scikit-learn con CountVectorizer, NumPy para operaciones matriciales y SciPy para matrices dispersas. NLTK o spaCy pueden sustituir nuestra tokenización sencilla cuando necesitamos reglas lingüísticas más completas.

Combinar regex y BoW en un mismo programa

Regex puede limpiar o transformar estructuras antes de construir el vocabulario. Aquí normalizamos un error conocido, sustituimos correos concretos por el token email y después sí construimos la matriz Bag of Words.

Regex + Bag of Words

```
import re

documentos = [
    "Correo: ana@example.com reportó un error.",
    "correoo soporte@example.com reportó errores.",
    "El CORREO de soporte resolvió otro error."
]

tokens_documentos = [None] * len(documentos)
indice = {}

i = 0

while i < len(documentos):

    texto = documentos[i].lower()

    texto = re.sub(
        r"\bcorreoo\b",
        "correo",
        texto
    )
```

```
    texto = re.sub(
        r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+",
        " email ",
        texto
    )

    tokens = re.findall(
        r"[a-záéíóúñ]+",
        texto
    )

    tokens_documentos[i] = tokens

    j = 0
    while j < len(tokens):
        palabra = tokens[j]
        if palabra not in indice:
            indice[palabra] = len(indice)
        j += 1

    i += 1

vocabulario = [None] * len(indice)

for palabra in indice:
    vocabulario[indice[palabra]] = palabra

matriz = [
    [0] * len(vocabulario)
    for _ in range(len(documentos))
]

i = 0
while i < len(tokens_documentos):
    j = 0
    while j < len(tokens_documentos[i]):
        palabra = tokens_documentos[i][j]
        matriz[i][indice[palabra]] += 1
        j += 1
    i += 1

print(vocabulario)

for i in range(len(matriz)):
    print("D", i + 1, matriz[i])
```

Salida:

```
['correo', 'email', 'reportó', 'un', 'error', 'errores', 'el', 'de', 'soporte', 'resolvió', 'otro']
D 1 [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]
D 2 [1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0]
D 3 [1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1]
```

Ahora sí vemos la combinación completa: regex modifica el texto y BoW cuenta el resultado. Los dos correos concretos terminan en una sola columna email. La variante correoo termina en la misma columna correo que la forma correcta.

BoW con librerías

CountVectorizer automatiza la tokenización, la creación del vocabulario y el conteo. Conceptualmente hace lo mismo que el programa anterior, pero está optimizado para corpus grandes y devuelve una matriz dispersa.

```
from sklearn.feature_extraction.text import CountVectorizer

documentos = [
    "El gato come pescado.",
    "El perro come carne.",
    "El gato y el perro juegan."
]

vectorizador = CountVectorizer(
    token_pattern=r"(?u)\b[a-záéíóúñ]+\b"
)
```

```
X = vectorizador.fit_transform(documentos)

print(vectorizador.get_feature_names_out())
print(X.toarray())
```

Salida:

```
['carne' 'come' 'el' 'gato' 'juegan' 'pescado' 'perro' 'y']
[[0 1 1 1 0 1 0 0]
 [1 1 1 0 0 0 1 0]
 [0 0 2 1 1 0 1 1]]
```

Repaso del patrón token_pattern

El valor token_pattern=r"(?u)\b[a-záéíóúñ]+\b" le dice a CountVectorizer qué secuencias debe considerar tokens. Conviene leerlo por partes:

Fragmento	Significado
`(?u)`	Activa explícitamente el modo Unicode del motor de expresiones regulares.
`\b`	Exige una frontera de palabra al inicio y al final.
`[a-záéíóúñ]`	Acepta letras minúsculas del rango indicado y vocales acentuadas/ñ.
`+`	Exige una o más letras consecutivas.

En Python 3 las cadenas de texto ya trabajan con Unicode por defecto, por lo que (?u) suele ser redundante; sin embargo, scikit-learn utiliza tradicionalmente patrones que lo incluyen de forma explícita. Su efecto es especialmente relevante para clases como \w, \b, \d y \s, porque les indica que deben interpretar caracteres Unicode y no únicamente ASCII. Aquí la clase [a-záéíóúñ] ya enumera letras concretas, pero \b también se beneficia de la semántica Unicode.

Importante. (?u) no significa una letra opcional ni un grupo de captura. Es una bandera inline del motor regex. Debe aparecer al inicio del patrón y equivale conceptualmente a activar el modo Unicode.

Qué parte del programa manual sustituye cada bloque de CountVectorizer

Código manual	CountVectorizer	Qué ocurre
`re.findall(...) para tokenizar`	token_pattern=...	Define qué secuencias cuentan como tokens.
if palabra not in indice	`fit(...)`	Construye el vocabulario y asigna columnas.
`matriz[i][columna] += 1`	`transform(...)`	Cuenta ocurrencias por documento.
vocabulario	`get_feature_names_out()`	Recupera el nombre de cada columna.
lista de listas	`X (sparse matrix)`	Guarda la matriz sin almacenar todos los ceros.

La versión manual resulta útil para aprender y controlar cada decisión. CountVectorizer es más eficiente para miles o millones de documentos porque utiliza estructuras optimizadas y matrices dispersas, evitando almacenar explícitamente una gran cantidad de ceros.

¿Para qué se puede utilizar BoW?

Aplicación	Cómo ayuda
Clasificación de comentarios	Cada documento se convierte en un vector para entrenar un clasificador.
Detección de spam	Palabras y expresiones frecuentes pueden servir como características.
Análisis de sentimiento clásico	Conteos alimentan modelos lineales.

Aplicación	Cómo ayuda
Clasificación temática	El vocabulario ayuda a distinguir temas por sus términos.
Búsqueda y similitud documental	Podemos comparar vectores entre documentos.
Enrutamiento de tickets	El texto del reporte puede convertirse en variables para predecir el área responsable.
Encuestas y respuestas abiertas	Podemos cuantificar vocabulario asociado con distintos grupos.

Por qué vale la pena dominar estas bases. En México el PLN tiene aplicaciones en banca, atención a clientes, clasificación documental, análisis de comentarios, búsqueda y asistentes. BoW y coocurrencia son técnicas clásicas, pero ayudan a comprender ideas que reaparecen en representaciones mucho más avanzadas.

Matriz de coocurrencia: pasar de qué aparece a qué aparece cerca

BoW organiza la información por documento. La coocurrencia cambia la pregunta: interesa saber qué palabras aparecen cerca unas de otras. Cada fila representa una palabra objetivo, cada columna una palabra de contexto y cada celda acumula cuántas veces se encontró esa relación.

Idea clave. BoW puede decirnos cuántas veces aparece modelo en D2. Coocurrencia puede decirnos con qué palabras aparece cerca modelo: funciona, error, datos, entrenamiento, entrega, etc.

Primer ejemplo: vecinos inmediatos

gato	come	pescado

Con la frase gato come pescado y distancia 1, solo se consideran vecinos inmediatos. Se obtienen las relaciones gato-come y come-pescado.

Coocurrencia con vecinos inmediatos

Fila \ Columna	gato	come	pescado
gato	0	1	0
come	1	0	1
pescado	0	1	0

La matriz es simétrica porque contamos la relación en ambas direcciones. gato aparece junto a come, por eso M[gato,come]=1 y M[come,gato]=1.

Segundo ejemplo

Frase. el alumno entrena modelo y el modelo entrena paciencia

el	alumno	entrena	modelo	y	paciencia

Par adyacente	Qué suma
el - alumno	+1 en ambas direcciones
alumno - entrena	+1 en ambas direcciones
entrena - modelo	aparece dos veces en la frase
modelo - y	+1
y - el	+1

Par adyacente	Qué suma
el - modelo	+1
entrena - paciencia	+1

Matriz del segundo ejemplo con distancia 1

Fila \ Columna	el	alumno	entrena	modelo	y	paciencia
el	0	1	0	1	1	0
alumno	1	0	1	0	0	0
entrena	0	1	0	2	0	1
modelo	1	0	2	0	1	0
y	1	0	0	1	0	0
paciencia	0	0	1	0	0	0

Por qué entrena-modelo vale 2. La secuencia aparece una vez como entrena modelo y después como modelo entrena. Como nuestra matriz no distingue dirección y contamos vecinos en ambos sentidos, la relación acumulada entre esas dos palabras llega a 2.

Tercer ejemplo: el mismo texto con distancia 2

Frase. el alumno entrena modelo y el modelo entrena paciencia

El texto es exactamente el mismo del segundo ejemplo. La única modificación es la distancia: ahora cada palabra central puede relacionarse con hasta dos posiciones anteriores y dos posteriores. Mantener la misma frase permite observar con claridad qué celdas aparecen o aumentan únicamente por ampliar la ventana.

Primero: numerar las posiciones

Posición	0	1	2	3	4	5	6	7	8
Palabra	el	alumno	entrena	modelo	y	el	modelo	entrena	paciencia

Con distancia 2, desde la posición i se revisan las posiciones i-2, i-1, i+1 e i+2 siempre que existan. La palabra central no se cuenta consigo misma. Cada vecino encontrado incrementa una celda de la fila correspondiente a la palabra central.

Cuatro ejemplos de cómo se desplaza la ventana

Ejemplo	Palabra central	Ventana válida	Pares que se agregan
1: i = 0	el (posición 0)	posiciones 1 y 2: alumno, entrena	el → alumno; el → entrena
2: i = 1	alumno (posición 1)	posiciones 0, 2 y 3: el, entrena, modelo	alumno → el; alumno → entrena; alumno → modelo
3: i = 2	entrena (posición 2)	posiciones 0, 1, 3 y 4: el, alumno, modelo, y	entrena → el; entrena → alumno; entrena → modelo; entrena → y
4: i = 5	el (posición 5)	posiciones 3, 4, 6 y 7: modelo, y, modelo, entrena	el → modelo; el → y; el → modelo; el → entrena

Atención al cuarto ejemplo. La palabra modelo aparece dos veces dentro de la ventana de la palabra el situada en la posición 5: una vez en la posición 3 y otra en la 6. Por ello, durante ese único paso la celda M[el, modelo] aumenta dos unidades. La matriz cuenta ocurrencias de asociaciones, no solamente si una relación existe o no.

Primera fotografía de la matriz: después de procesar i = 0 e i = 1

Hasta este momento únicamente se han procesado las palabras centrales el y alumno. Por eso solo sus filas contienen incrementos. Las demás filas continúan en cero, aunque algunas de esas palabras ya hayan aparecido como contexto.

Matriz parcial después de las posiciones 0 y 1

Fila \ Columna	el	alumno	entrena	modelo	y	paciencia
el	0	1	1	0	0	0
alumno	1	0	1	1	0	0
entrena	0	0	0	0	0	0
modelo	0	0	0	0	0	0
y	0	0	0	0	0	0
paciencia	0	0	0	0	0	0

Segunda fotografía: después de procesar hasta i = 3

Al incorporar entrena de la posición 2 y modelo de la posición 3 aparecen nuevas asociaciones. Por ejemplo, entrena queda relacionada con el aunque exista alumno entre ambas palabras; esa relación no aparecía con distancia 1.

Matriz parcial después de las posiciones 0, 1, 2 y 3

Fila \ Columna	el	alumno	entrena	modelo	y	paciencia
el	0	1	1	0	0	0
alumno	1	0	1	1	0	0
entrena	1	1	0	1	1	0
modelo	1	1	1	0	1	0
y	0	0	0	0	0	0
paciencia	0	0	0	0	0	0

Asociación nueva o acumulada	Qué ocurrió
entrena → el	Aparece porque el está a dos posiciones de entrena en i = 2.
entrena → y	Aparece porque y está a dos posiciones de entrena en i = 2.
modelo → alumno	Aparece porque alumno está a dos posiciones de modelo en i = 3.
modelo → el	Aparece porque el de la posición 5 está a dos posiciones de modelo en i = 3.

Matriz final con distancia 2

Después de desplazar la palabra central por las nueve posiciones, todos los incrementos quedan acumulados. Las celdas con valor 2 indican que la misma pareja se encontró dos veces dentro de ventanas válidas durante el recorrido.

Matriz del tercer ejemplo con distancia 2

Fila \ Columna	el	alumno	entrena	modelo	y	paciencia
el	0	1	2	2	1	0
alumno	1	0	1	1	0	0
entrena	2	1	0	2	1	1
modelo	2	1	2	0	2	1
y	1	0	1	2	0	0
paciencia	0	0	1	1	0	0

Relación	Distancia 1	Distancia 2	Interpretación
el - entrena	0	2	No eran vecinos inmediatos; con distancia 2 aparecen dos asociaciones.
alumno - modelo	0	1	La palabra entrena queda entre ambas; distancia 2 permite relacionarlas.
modelo - paciencia	0	1	Paciencia queda a dos posiciones de la segunda aparición de modelo.
modelo - y	1	2	La ventana mayor permite observar esa asociación alrededor de más posiciones.
entrena - modelo	2	2	Ya eran vecinos inmediatos; ampliar la distancia no aumenta este conteo concreto.

Diferencia principal. Con distancia 1 solo se registran relaciones inmediatamente adyacentes. Con distancia 2 también se registran palabras separadas por un término intermedio. Esto crea nuevas celdas distintas de cero y puede aumentar conteos existentes. Seguir el recorrido posición por posición permite comprender de dónde sale cada número de la matriz final.

Para seguir con atención. Una matriz de coocurrencia se vuelve mucho más sencilla cuando se interpreta como el registro acumulado de pequeñas decisiones locales. Conviene comprobar primero cuatro o cinco ventanas a mano y después dejar que el programa haga el recorrido completo. Este razonamiento aparece constantemente en PLN y dominarlo facilita temas posteriores.

Otro ejemplo con distancia 2

Con la frase el profe explica regex y el alumno pregunta otra vez, una distancia 2 permite relacionar cada palabra con hasta dos posiciones anteriores y dos posteriores.

Palabra central	Vecinos con distancia 2
profe	el, explica, regex
explica	el, profe, regex, y
regex	profe, explica, y, el
alumno	y, el, pregunta, otra
pregunta	el, alumno, otra, vez

Qué cambia. Con distancia 2 aparecen relaciones que no eran adyacentes. profe-regex ahora cuenta como coocurrencia porque hay una sola palabra entre ambas. Ganamos contexto, pero también incorporamos relaciones menos locales.

Construcción en Python

La matriz se construye sin set, sorted, enumerate, append, split ni join. re.findall() funciona como tokenizador. Además, la lógica se separa en dos funciones: construir_indice() asigna columnas y construir_coocurrencia() llena la matriz según la distancia. Esta separación permitirá reutilizar exactamente la misma función más adelante.

Coocurrencia desde cero con distancia 1

```
import re


def construir_indice(tokens):
```

```
    indice = {}

    i = 0

    while i < len(tokens):

        palabra = tokens[i]

        if palabra not in indice:
            indice[palabra] = len(indice)

        i += 1

    return indice


def construir_coocurrencia(tokens, indice, distancia):

    n = len(indice)

    matriz = [
        [0] * n
        for _ in range(n)
    ]

    i = 0

    while i < len(tokens):

        inicio = i - distancia

        if inicio < 0:
            inicio = 0

        fin = i + distancia

        if fin >= len(tokens):
            fin = len(tokens) - 1

        j = inicio

        while j <= fin:

            if i != j:

                fila = indice[tokens[i]]
                columna = indice[tokens[j]]

                matriz[fila][columna] += 1

            j += 1

        i += 1

    return matriz


texto = """
el profe explica regex
y el alumno pregunta otra vez
"""

tokens = re.findall(
    r"[a-záéíóúñ]+",
    texto.lower()
)

indice = construir_indice(tokens)

vocabulario = [None] * len(indice)

for palabra in indice:
    vocabulario[indice[palabra]] = palabra
```

```
matriz = construir_coocurrencia(
    tokens,
    indice,
    distancia=1
)

print(vocabulario)

i = 0

while i < len(matriz):

    print(
        vocabulario[i],
        matriz[i]
    )

    i += 1
```

Salida:

```
['el', 'profe', 'explica', 'regex', 'y', 'alumno', 'pregunta', 'otra', 'vez']
el [0, 1, 0, 0, 1, 1, 0, 0, 0]
profe [1, 0, 1, 0, 0, 0, 0, 0, 0]
explica [0, 1, 0, 1, 0, 0, 0, 0, 0]
regex [0, 0, 1, 0, 1, 0, 0, 0, 0]
y [1, 0, 0, 1, 0, 0, 0, 0, 0]
alumno [1, 0, 0, 0, 0, 0, 1, 0, 0]
pregunta [0, 0, 0, 0, 0, 1, 0, 1, 0]
otra [0, 0, 0, 0, 0, 0, 1, 0, 1]
vez [0, 0, 0, 0, 0, 0, 0, 1, 0]
```

Dentro de construir_coocurrencia(), el ciclo exterior selecciona la palabra central y el ciclo interior recorre las posiciones de su ventana. Cuando i y j son iguales no se cuenta nada, porque correspondería a la palabra consigo misma. El parámetro distancia permite reutilizar la misma función para ventanas de 1, 2 o más posiciones.

Una versión más eficiente con NumPy

Cuando la matriz crece, el almacenamiento numérico puede delegarse a NumPy. La lógica permanece igual; cambia la estructura usada para guardar los conteos. En este caso la distancia aumenta a 2 para observar cómo cambia la salida.

Misma lógica, matriz NumPy y distancia 2

```
import re
import numpy as np

texto = """
el profe explica regex
y el alumno pregunta otra vez
"""

tokens = re.findall(
    r"[a-záéíóúñ]+",
    texto.lower()
)

indice = {}

i = 0
while i < len(tokens):
    palabra = tokens[i]
    if palabra not in indice:
        indice[palabra] = len(indice)
    i += 1

vocabulario = [None] * len(indice)

for palabra in indice:
    vocabulario[indice[palabra]] = palabra

matriz = np.zeros(
    (len(vocabulario), len(vocabulario)),
    dtype=np.int32
```

```
)

distancia = 2

i = 0
while i < len(tokens):

    inicio = max(0, i - distancia)
    fin = min(len(tokens) - 1, i + distancia)

    j = inicio

    while j <= fin:

        if i != j:
            fila = indice[tokens[i]]
            columna = indice[tokens[j]]
            matriz[fila, columna] += 1

        j += 1

    i += 1

print(vocabulario)
print(matriz)
```

Salida:

```
['el', 'profe', 'explica', 'regex', 'y', 'alumno', 'pregunta', 'otra', 'vez']
[[0 1 1 1 1 1 1 0 0]
 [1 0 1 1 0 0 0 0 0]
 [1 1 0 1 1 0 0 0 0]
 [1 1 1 0 1 0 0 0 0]
 [1 0 1 1 0 1 0 0 0]
 [1 0 0 0 1 0 1 1 0]
 [1 0 0 0 0 1 0 1 1]
 [0 0 0 0 0 1 1 0 1]
 [0 0 0 0 0 0 1 1 0]]
```

Versión desde cero	Versión con NumPy	Equivalencia conceptual
lista de listas	`np.zeros(...)`	Reserva la matriz numérica.
`matriz[fila][columna] += 1`	`matriz[fila, columna] += 1`	Incrementa una relación.
límites con if	`max(...) y min(...)`	Limita la ventana al inicio/final.
impresión fila por fila	`print(matriz)`	Muestra el arreglo completo.

NumPy no elimina el costo de recorrer ventanas, pero almacena y opera la matriz de forma mucho más eficiente que listas puras. Para vocabularios grandes, SciPy sparse evita almacenar millones de ceros.

Librerías útiles. NumPy sirve para matrices densas pequeñas o medianas; SciPy sparse para matrices dispersas grandes; pandas para inspección; NLTK o spaCy para tokenización y lematización; Gensim para pasos posteriores de representaciones distribucionales.

Ejemplo más grande: regex + coocurrencia

En este caso se combinan regex y coocurrencia de forma completa. Antes del código conviene observar por qué la asociación entre palabras es útil: una etiqueta como FOLIO, aislada, solo informa que existe un folio; su fila de coocurrencia muestra qué términos aparecen cerca de esa estructura. Si alrededor de FOLIO se repiten error, revisar, reporte o soporte, la matriz empieza a describir el contexto en el que ese tipo de dato aparece.

Regex + matriz de coocurrencia

```
import re


def normalizar_estructuras(texto):
```

```
    texto = texto.lower()

    # Correos -> email
    texto = re.sub(
        r"[\w.+-]+@[\w-]+(?:\.[\w-]+)+",
        " email ",
        texto
    )

    # Folios -> folio
    texto = re.sub(
        r"\b[a-z]{3}-\d{4}-\d{5}\b",
        " folio ",
        texto
    )

    # Fechas -> fecha
    texto = re.sub(
        r"\b\d{2}/\d{2}/\d{4}\b",
        " fecha ",
        texto
    )

    # Montos -> monto
    texto = re.sub(
        r"\$\s*\d+(?:,\d{3})*(?:\.\d{2})?",
        " monto ",
        texto
    )

    return texto


texto = """
El alumno escribió a soporte@escom.mx porque
ABC-2026-00451 marcaba error el 21/10/2026.

Después comentó que el modelo funcionaba perfecto
hasta cinco minutos antes de la entrega.

Soporte respondió desde ayuda@escom.mx y pidió revisar
PLN-2026-00007.

El reporte también menciona un costo de $1,250.50
y el 22/10/2026 como nueva revisión.

Al final el alumno dijo que si compila a la primera
seguramente olvidó guardar algo.
"""

texto_normalizado = normalizar_estructuras(texto)

print(texto_normalizado)
```

Paso 1: convertir variantes concretas en categorías. Dos direcciones de correo diferentes se transforman en email; dos folios diferentes se transforman en folio. De esta manera, la matriz no dispersa el contexto entre identificadores únicos cuando la pregunta de análisis se refiere a la categoría general.

Bloque 2. Tokenización, índice y reutilización de la función anterior

```
tokens = re.findall(
    r"[a-záéíóúñ]+",
    texto_normalizado
)

# Función definida en el ejemplo anterior
indice = construir_indice(tokens)

vocabulario = [None] * len(indice)

for palabra in indice:
    vocabulario[indice[palabra]] = palabra
```

```
# Se reutiliza exactamente la misma función
matriz = construir_coocurrencia(
    tokens,
    indice,
    distancia=2
)
```

Paso 2: la matriz convierte cercanía en números. Cada fila se convierte en un perfil de contexto. La fila de folio acumula las palabras que aparecen a dos posiciones de cada aparición de folio; la fila de monto hace lo mismo para monto. Las asociaciones repetidas producen valores mayores.

Bloque 3. Consultar únicamente las filas de interés

```
objetivos = [
    "email",
    "folio",
    "fecha",
    "monto"
]

i = 0

while i < len(objetivos):

    objetivo = objetivos[i]
    fila = matriz[indice[objetivo]]

    print("\n", objetivo.upper())

    j = 0

    while j < len(vocabulario):

        if fila[j] > 0:
            print(
                vocabulario[j],
                fila[j]
            )

        j += 1

    i += 1
```

Lectura de la asociación. Si folio aparece cerca de error, revisar y reporte, esos términos forman parte de su contexto en este corpus. La asociación no demuestra causalidad ni significado por sí sola, pero proporciona una señal cuantitativa que puede compararse entre documentos, periodos o grupos.

```
EMAIL
a 1
escribió 1
porque 1
folio 1
respondió 1
desde 1
y 1
pidió 1

FOLIO
el 1
email 1
porque 1
marcaba 1
error 1
pidió 1
revisar 1
reporte 1

FECHA
el 2
error 1
después 1
```

```
comentó 1
y 1
como 1
nueva 1

MONTO
de 1
costo 1
y 1
el 1
```

Salida:

La salida ya no es solo una lista de tokens: cada etiqueta dispone de un vector de contexto. Por ejemplo, folio queda asociado con términos como error, revisar y reporte dentro de la distancia definida.

Decisión metodológica. Si necesitamos saber qué correo concreto fue usado o qué folio específico tuvo el error, no debemos sustituirlos. Si nuestra pregunta es más general -por ejemplo, qué palabras rodean a un folio o a un monto-, las etiquetas reducen la dispersión.

Aplicaciones de la matriz de coocurrencia

Aplicación	Qué se puede observar
Vocabulario de un dominio	Qué palabras rodean a fraude, bloqueo, entrega, error o campaña.
Expansión de términos	Encontrar palabras con contextos parecidos a un término semilla.
Análisis exploratorio	Detectar asociaciones frecuentes que no habíamos previsto.
Colocaciones	Identificar combinaciones que aparecen juntas con frecuencia.
Similitud distribucional	Comparar filas de la matriz como vectores de contexto.
Ingeniería de características	Crear variables de contexto para modelos posteriores.
Análisis temporal o por grupo	Comparar coocurrencias entre meses, productos, clases o segmentos.

Comparación final: One-Hot, BoW y coocurrencia

Aspecto	One-Hot	Bag of Words	Coocurrencia
Unidad principal	Palabra/categoría	Documento	Palabra objetivo
Filas típicas	Palabra	Documento	Palabra
Columnas	Vocabulario	Vocabulario	Contexto
Valor	0 o 1	Conteo o ponderación	Número de apariciones cercanas
Frecuencia documental	No	Sí	No como objetivo principal
Contexto local	No	No en BoW clásico	Sí
Orden global	No	No	No
Uso natural	Identidad/categorías	Clasificación documental	Relaciones entre términos
Forma típica	palabra x vocabulario	documento x vocabulario	palabra x palabra

La misma frase vista de tres maneras

Con el vocabulario [modelo, funciona, entrega], la palabra modelo puede representarse como [1,0,0] en One-Hot. El documento 'modelo funciona entrega' puede ser [1,1,1] en BoW. Y, si funciona es la única palabra inmediatamente vecina de modelo, la fila de coocurrencia de modelo puede ser [0,1,0].

One-Hot: quién es 'modelo'

Fila \ Columna	modelo	funciona	entrega
modelo	1	0	0

BoW: qué contiene el documento

Fila \ Columna	modelo	funciona	entrega
Documento	1	1	1

Coocurrencia: quién aparece cerca de 'modelo'

Fila \ Columna	modelo	funciona	entrega
modelo	0	1	0

Resumen conceptual. One-Hot indica QUIÉN es un término. BoW indica QUÉ contiene cada documento. Coocurrencia indica CON QUÉ aparece una palabra. TF-IDF mantiene la estructura de BoW, pero sustituye conteos simples por pesos de importancia relativa.

Último empujón. BoW y coocurrencia parecen sencillos porque sus operaciones son conteos, pero son ideas fundacionales de PLN. Entenderlas evita memorizar matrices sin sentido y, de paso, ayuda a mantener la materia del lado de las materias aprobadas.

Ejercicios sugeridos

1. Construyan un BoW con tres frases propias del grupo y expliquen cada columna de la matriz.
2. Programen el BoW desde cero usando regex para tokenizar y un diccionario para asignar columnas.
3. Normalicen correos, folios, fechas y montos mediante regex antes de construir un BoW.
4. Construyan una coocurrencia con distancia 1 y después con distancia 2; comparen qué relaciones nuevas aparecen.
5. Repitan la coocurrencia usando NumPy y expliquen qué parte reemplaza a la matriz de listas.
6. Elijan una palabra objetivo del corpus y analicen sus contextos más frecuentes.
7. Comparen qué problema resolverían con BoW y cuál con coocurrencia usando un conjunto de comentarios reales o simulados.

Idea final

El objetivo no es memorizar matrices, sino entender qué información conserva cada representación. Para representar documentos, BoW es un punto de partida sólido. Para estudiar relaciones entre palabras, la coocurrencia ofrece una visión contextual. Antes de ambas, regex puede limpiar, normalizar y convertir estructuras concretas en tokens coherentes.

Texto crudo → **Regex** → **Tokens** → **BoW** → **Coocurrencia** → **Modelo o análisis**

Fin del apunte